A

Project Report

on

BUILD SLAVES AND BUILD PHASES IN DEVOPS

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

P.JOSHIKA

24K81A05P9

Under the guidance of

Mr.M.DILEEP KUMAR

Assistant professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

NBA& NAAC A+ Accredited Dhulapally, Secunderabad - 500 100

www.smec.ac.in

Certificate

This is to certify that the project entitled " **BUILD SLAVES AND BUILD PHASES IN DEVOPS**" is being submitted by **P.JOSHIKA (24K81A05P9)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of Bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner

Mr. M. DILEEP KUMAR

Assistant professor

Department of CSE

Signature of HOD

Mr. D. KRISHNA KISHORE

Associate Professor and HOD

Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

NBA& NAAC A+ Accredited Dhulapally, Secunderabad - 500 100

www.smec.ac.in

COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of "**Bachelor of Technology in Department of Computer Science and Engineering"**, session: 2024-2028, **St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad** , here by declare that the work presented in this project work is **BUILD PHASES AND BUILD SLAVES** is the outcome of our own Bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

P.JOSHIKA -24K81A05P9

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express my deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express my deep sense of gratitude and indebtedness to **Dr. K. RAVINDRA,** Director, Principal, Professor St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

I am also thankful **Mr.D. KRISHNA KISHORE,** Head of the Department of Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout my project.

I would like to express my sincere gratitude,thanks and indebtedness to my project supervisor **Mr. M. DILEEP KUMAR** Assistant Professor, Department of Computer Science and Engineering, St. Martins Engineering College, Dhulapally, for his support and guidance throughout project.

Finally, I express thanks to all those who have helped me in successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

P.JOSHIKA -24K81A05P9

CONTENTS

1. ABSTRACT **1**

2. INTRODUCTION **2**

3. LITERATURE SURVEY **3**

4. SYSTEM REQUIREMENT **4**

5. FLOW CHART **5**

6. ALGORITHM **6**

7. SOURCE CODE **7**

8. EXPERIMENTAL RESULTS **12**

9. CONCLUSION **13**

10. FUTURE ENHANCEMENT **14**

11. REFERENCES **15**

1. ABSTRACT

The project titled **"BUILD SLAVES AND BUILD PHASES IN DEVOPS"** focuses on understanding and implementing automated software build processes using Jenkins, a popular continuous integration and continuous delivery (CI/CD) tool. Build slaves, also known as build agents, are responsible for executing build tasks assigned by the Jenkins controller, helping distribute workloads and improve efficiency. Build phases organize the software development process into stages such as source code checkout, building, testing, packaging, and deployment. This project demonstrates how Jenkins pipelines automate these stages and how build agents execute assigned tasks. The main objective is to reduce manual effort, improve build consistency, identify errors early, and make the software development process faster and more reliable

2. INTRODUCTION

DevOps is a software development approach that combines Development and Operations to improve collaboration, automation, and the delivery of software applications. It helps organizations develop, test, and release software efficiently by integrating automated tools and continuous integration and continuous delivery (CI/CD) practices. Jenkins is one of the widely used automation tools in DevOps that enables developers to create and manage automated build pipelines.Build slaves, also called build agents, are machines or environments that execute tasks assigned by the Jenkins controller. They help distribute workloads, support parallel execution, and reduce the processing burden on the controller.

Build phases represent the different stages involved in the software build process, including source code checkout, compilation, testing, packaging, and deployment. By combining build agents with automated build phases, Jenkins helps ensure that software is built and tested consistently. This project demonstrates the role of build slaves and build phases in automating software development workflows, improving productivity, and reducing human errors.

Furthermore, the use of build slaves and build phases helps simplify the management of complex software projects by dividing tasks into smaller, organized stages. Build agents can be configured to handle specific workloads based on the requirements of the project, allowing multiple jobs to run efficiently across different machines. Jenkins pipelines provide a structured way to define these stages and monitor their execution through console logs and build status reports. If any stage fails, developers can identify the problem and take corrective action before proceeding further. This approach improves transparency, saves development time, and supports continuous integration practices. Therefore, understanding build slaves and build phases is essential for implementing an efficient, scalable, and automated DevOps environment.

UGC AUTONOMOUS

3.L ITERATURE SURVEY

The literature survey on Build Slaves and Build Phases in DevOps focuses on distributed build execution, pipeline automation, continuous integration, and software delivery. Various tools and approaches have been developed to automate repetitive tasks, distribute workloads, and improve the reliability of software development processes.

1. **Jenkins Build Agents**

Jenkins supports distributed build execution through a controller-agent architecture. The controller schedules and manages jobs, while build agents execute assigned tasks. Agents can run on separate computers or environments, allowing workloads to be distributed according to available resources. This approach is useful for projects that require multiple builds or different execution environments**.**

2. Build Phases and Pipeline Automation

Build phases divide the software development workflow into organized stages. Common stages include source code checkout, compilation, testing, packaging, and deployment. Jenkins Pipeline allows developers to define these stages in a `Jenkinsfile`, making the workflow repeatable and easier to monitor. Pipeline automation reduces manual intervention and helps identify errors during execution.

3. Continuous Integration and Continuous Delivery

Continuous Integration (CI) involves regularly integrating code changes and checking them through automated builds and tests. Continuous Delivery (CD) extends this process by preparing software for release through automated workflows. Jenkins supports these practices by executing pipeline stages whenever a build is triggered. Combining CI/CD with build agents helps improve consistency and supports faster software delivery.

4. Build Monitoring and Error Handling

Monitoring build execution is an important part of automated software development. Jenkins provides console output, build history, stage status, and error messages to help developers understand the outcome of each stage. If a build or test fails, the pipeline can report the failure and stop subsequent stages when configured to do so. These features help developers identify problems early and improve the reliability of the build process.

4. SYSTEM REQUIREMENT

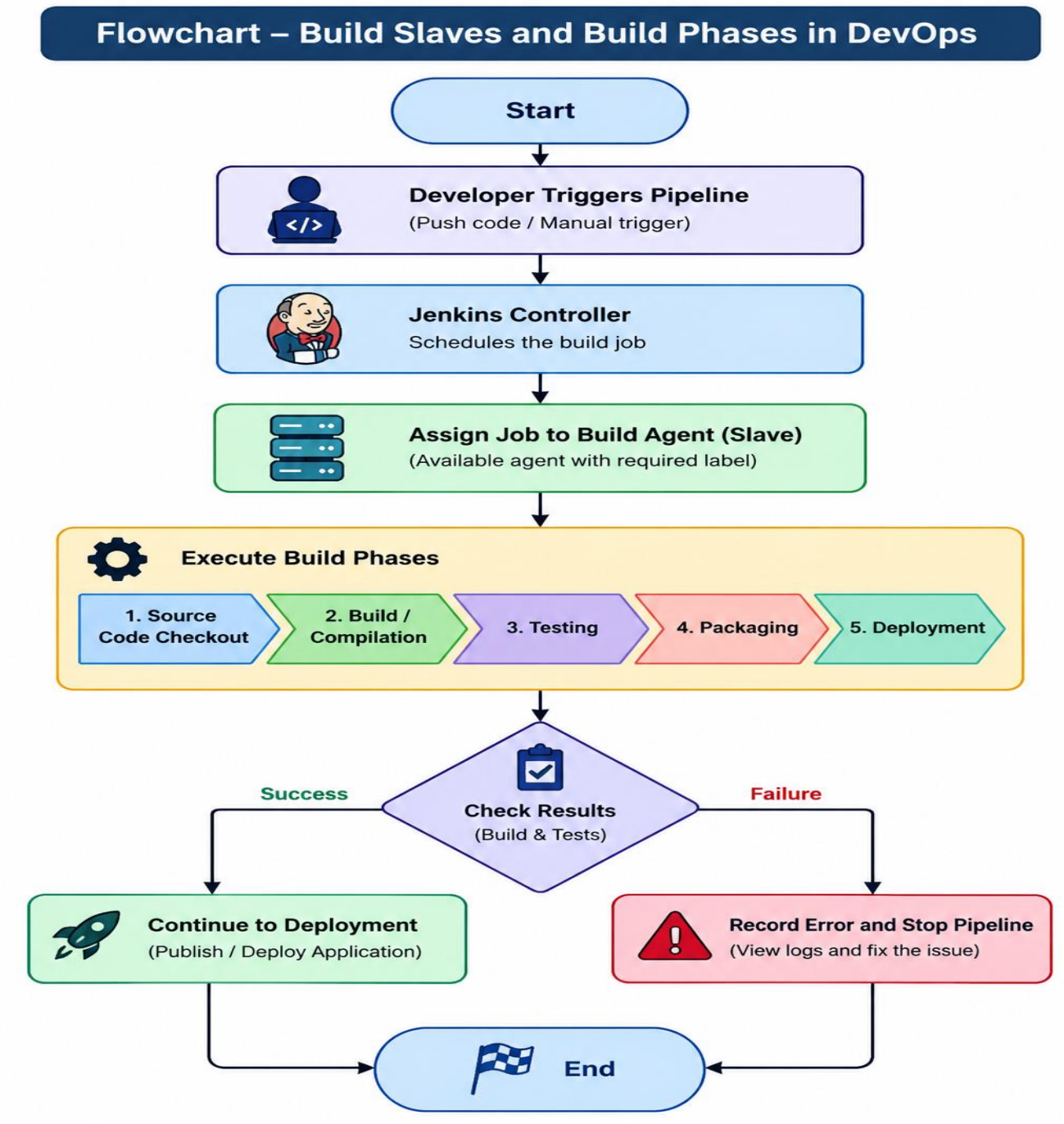
1. Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	10 GB free space
System Type	64-bit computer
Network	Internet or local network connection

2. Software Requirements

Software	Requirement
Operating System	Windows 10/11, Linux, or macOS
Automation Tool	Jenkins
Programming / Scripting	Groovy and Shell scripting
Build Agent	Jenkins agent environment
Version Control	Git
Pipeline Definition	jenkinsfile
Browser	Chrome,Edge or Firefox
java	Compatible Java version required by the installed Jenkins
Source Code Repository	GitHub or a local Git repository

5. FLOW CHART


Flowchart – Build Slaves and Build Phases in DevOps
Start
Developer Triggers Pipeline
(Push code / Manual trigger)
Jenkins Controller
Schedules the build job
Assign Job to Build Agent (Slave)
(Available agent with required label)
Execute Build Phases
1. Source Code Checkout
2. Build / Compilation
3. Testing
4. Packaging
5. Deployment
Check Results
(Build & Tests)
Success
Failure
Continue to Deployment
(Publish / Deploy Application)
Record Error and Stop Pipeline
(View logs and fix the issue)
End

6. ALGORITHM

1. Start the process.

2. Configure Jenkins and create a build agent.

3. Connect the build agent to the Jenkins controller.

4. Create a Jenkins pipeline using a `Jenkinsfile`.

5. Trigger the pipeline manually or through a configured automation mechanism.

6. Allow the Jenkins controller to schedule the job on an eligible build agent.

7. Retrieve the source code from the configured repository, if one is used.

8. Execute the build phase using the required build commands.

9. Run automated tests to check the correctness of the application.

10. Check whether the build and tests are successful.

- If successful, proceed to the packaging phase.
- If unsuccessful, record the error and mark the pipeline as failed.

11. Package the application or generate the required build artifact.

12. Execute the deployment phase if deployment is configured.

13. Display the pipeline status and execution logs in Jenkins.

14. End the process.

7. SOURCE CODE

```
pipeline {
    agent {
        label 'build-slave'
    }

    stages {

        stage('Source Code Checkout') {
            steps {
                echo 'Phase 1: Starting source code
    checkout'
                checkout scm
                echo 'Source code checkout
    completed'
            }
        }

        stage('Build Phase') {
            steps {
                echo 'Phase 2: Building the
    application'

                sh '''
                    echo "Build Slaves and Build
    Phases Project" > app.txt
                    echo "Application build completed"
                '''
            }
        }

        stage('Testing Phase') {
            steps {
                echo 'Phase 3: Testing the application'
```

```
sh '''
                test -s app.txt
                echo "Test passed: Application file
  exists"
            '''
        }
    }

    stage('Packaging Phase') {
        steps {
            echo 'Phase 4: Packaging the
  application'

            sh '''
                tar -czf application.tar.gz app.txt
                echo "Application packaged
  successfully"
            '''
        }

        post {
            success {
                archiveArtifacts artifacts:
  'application.tar.gz'
            }
        }
    }

    stage('Deployment Phase') {
        steps {
            echo 'Phase 5: Deployment
  demonstration'
            echo 'Deployment phase completed'
        }
    }
  }
```

```
    post {
        success {
            echo 'Pipeline completed successfully!'
        }

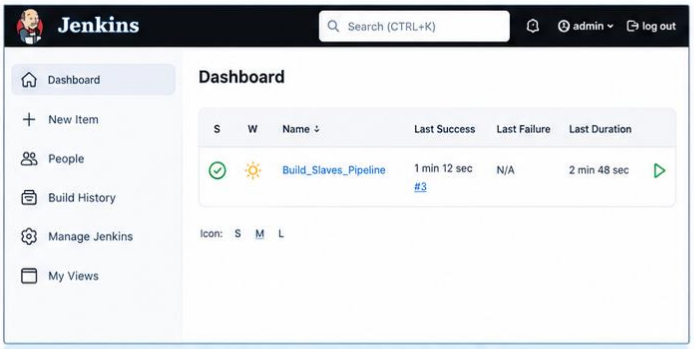
        failure {
            echo 'Pipeline failed. Check the console
    logs.'
        }

        always {
            echo 'Build execution finished.'
        }
    }
}except requests.RequestException as error:
    print("SOAP request failed:", error)

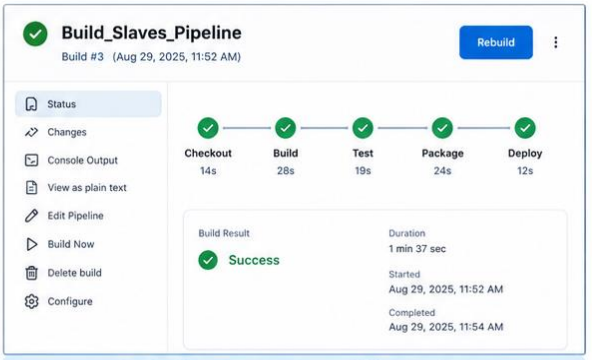
except ET.ParseError:
    print("Error: Invalid XML response.")</html>
```

8. EXPERIMENTAL RESULTS

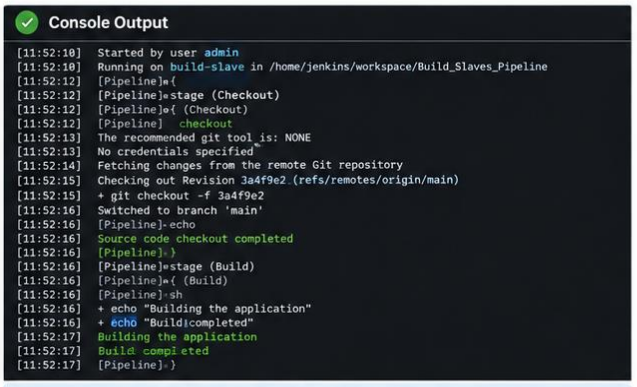
Experimental Results – Build Slaves and Build Phases in DevOps



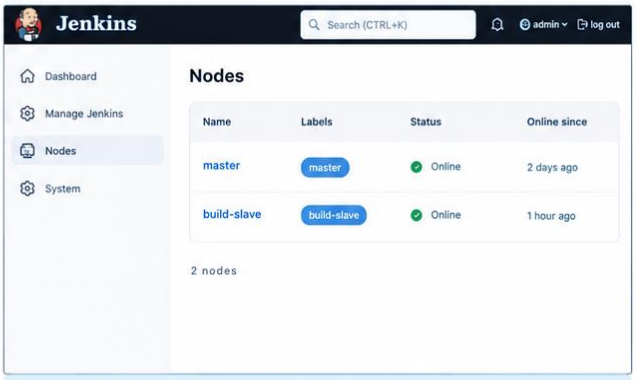
1. Jenkins Dashboard – Pipeline Job



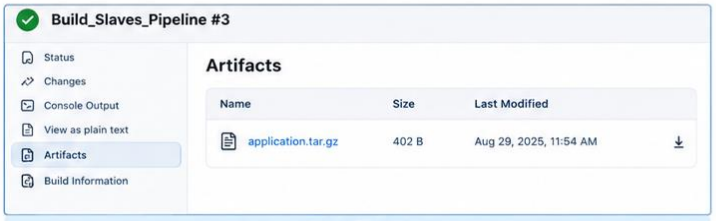
2. Pipeline Stages – Successful Execution



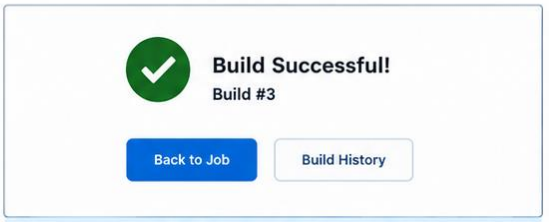
3. Console Output – Build and Test Phases



4. Build Agent (Slave) Configuration



5. Retrieved Build Artifact



6. Final Output – Build Successful

UGC AUTONOMOUS

9. CONCLUSION

The project “Build Slaves and Build Phases in DevOps” explains the importance of distributed build execution and pipeline automation in software development. It demonstrates how Jenkins uses a controller and build agents to manage and execute automated tasks. The project also explains the role of different build phases, including source code checkout, building, testing, packaging, and deployment.

The Jenkins Pipeline implementation demonstrates how these stages can be organized and executed in a predefined sequence. It creates an application file, validates the generated file, packages it into an archive, and displays the execution status. Jenkins also provides console logs and build results that help identify errors and monitor the progress of the pipeline. The demonstration illustrates how automated build processes can reduce repetitive manual work and improve consistency.

Overall, this project provides a practical understanding of build agents and pipeline stages in a DevOps environment. By combining distributed execution with automated build phases, Jenkins helps make software development workflows more organized, repeatable, and reliable. The project also provides a foundation for learning advanced CI/CD practices and implementing more complete software deployment workflows.

.

10. FUTURE ENHANCEMENTS

1. Automated Deployment: The project can be extended to deploy applications automatically to a test server or cloud platform after successful testing and packaging.

2. Multiple Build Agents: Additional agents can be configured to distribute workloads and execute different build jobs across multiple machines.

3. Automated Testing: Unit tests, integration tests, and code-quality checks can be added to identify errors and improve software reliability.

4. GitHub Integration: Webhooks can be configured to trigger Jenkins pipelines automatically whenever developers push new code to a repository.

5. Docker Integration: Docker containers can provide consistent build environments and help applications run reliably across different systems.

6. Parallel Build Execution: Independent stages or jobs can be configured to run in parallel where appropriate, reducing total execution time.

7. Build Notifications: Email or messaging notifications can be added to inform developers about successful builds and failures.

8. Security Improvements: Credentials management, role-based access control, and secure communication can be implemented to protect Jenkins and its agents.

9. Performance Monitoring: Build duration, agent utilization, queue time, and failure rates can be monitored to identify opportunities for optimization.

10. Cloud-Based Build Agents: Cloud infrastructure can be used to create temporary agents when workloads increase, helping improve scalability and resource utilization.

11. REFERENCES

- Jenkins Official Documentation — Jenkins User Documentation.
- Jenkins Official Documentation — Using Jenkins Agents.
- Jenkins Official Documentation — Pipeline Syntax.
- Jenkins Official Documentation — Using a Jenkinsfile.
- Jenkins Official Documentation — Pipeline Steps Reference.
- Git Official Documentation — Git Reference Manual.
- Apache Maven — Maven Getting Started Guide.
- Docker Documentation — Docker Overview.
- Atlassian — Continuous Integration and Continuous Delivery.
- GitHub Documentation — About Continuous Integration

1.